

Deep Learning Foundations

From perceptrons to LSTMs.

1. The Biological Inspiration

The Perceptron (1950s)

The original artificial neuron.

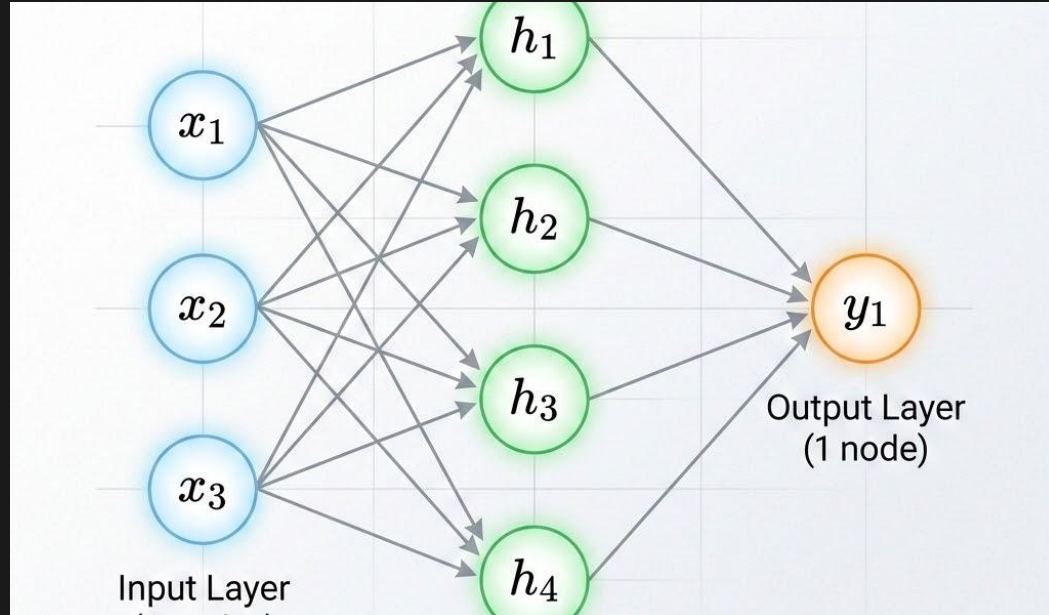
- Binary step function output.
- Hard threshold (exactly 0 or 1).
- Non-differentiable (gradient is zero or undefined).
- Cannot be trained via backpropagation.

Modern Neurons

The foundation of modern deep learning.

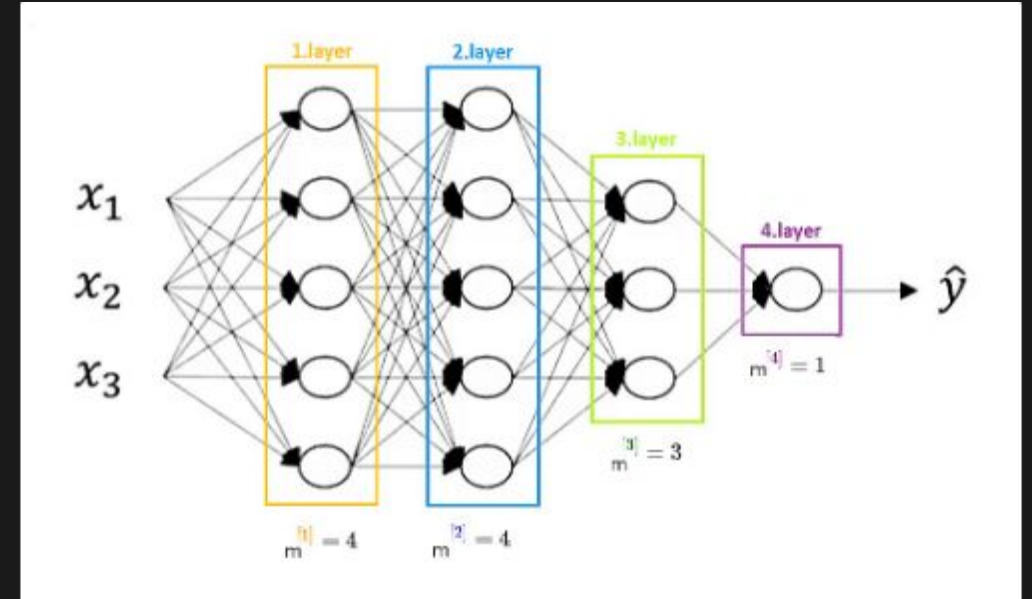
- Smooth activation functions (Sigmoid, ReLU).
- Outputs probability distributions or continuous values.
- Fully differentiable curves.
- Enables calculus-based optimization.

2. Neural Network



Shallow Architecture

A classic 3-layer Multilayer Perceptron: Input Layer, a single Hidden Layer, and the Output Layer.



Deep Architecture

A 4-layer Deep Neural Network: The introduction of multiple consecutive hidden layers allows for hierarchical feature extraction.

3. Gradient Descent

The Objective

To mathematically minimize the neural network's loss function by iteratively updating model parameters based on the calculated error.

The Mechanism

Calculating the slope (gradient) of the multidimensional error curve at the network's current position to determine the direction of steepest descent.

The Step

Adjusting the network weights proportionally in the exact opposite direction of the calculated slope, scaled by the learning rate.

The Goal

Iteratively stepping down the loss landscape over hundreds of epochs until global (or acceptable local) convergence is reached.

4. Optimization: SGD vs GD

Batch Gradient Descent (GD)

Processes the entire training dataset before updating the weights.

- Computes the exact gradient for maximum accuracy.
- Produces a smooth, stable descent path.
- Highly memory-intensive on large datasets.
- Prone to getting permanently stuck in shallow local minima.

Stochastic Gradient Descent (SGD)

Updates the network weights using a single sample (or mini-batch).

- Computes a noisy, approximate gradient.
- Dramatically faster iteration and training speed.
- Low memory requirements.
- Inherent noise helps the model escape local minima.

5. Visualizing Optimization

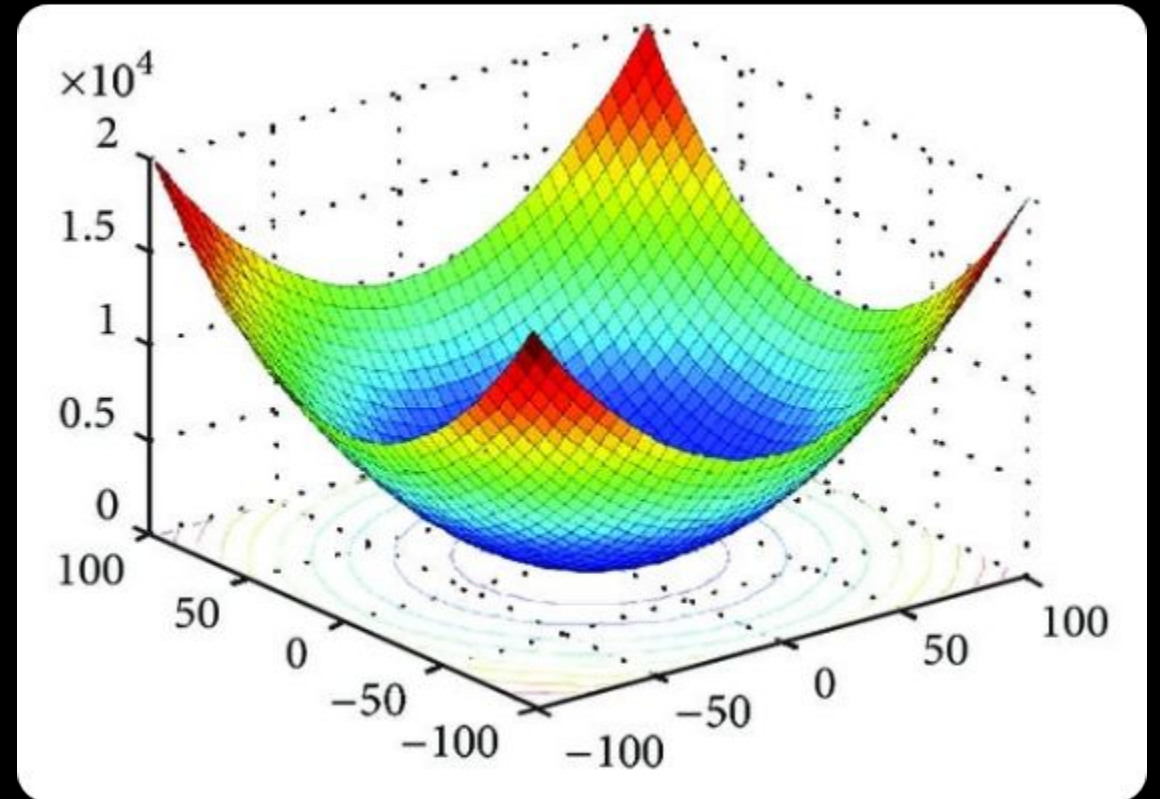
The Loss Landscape

The graph represents a 3D convex bowl, visualizing the loss function. The network starts at a random high point (high error) and uses the slope to iteratively step down into the valley (minimum error).

The Formula

$$W_{\text{new}} = W_{\text{old}} - \alpha \frac{\partial L}{\partial W}$$

- **W**: Model Weights
- **α**: Learning Rate (Step Size)
- **∂L/∂W**: Partial Derivative (Slope)



6. Backpropagation & Calculus

Error Propagation

- Passes error backwards through the layers.
- Calculates partial derivatives for connections.

1. Equation in the output layer:

$$\delta^L = \nabla_a C \odot \sigma' (z^L)$$

2. Equation in terms of next layer:

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma' (z^l)$$

Gradient Calculation

Translating the error terms into precise adjustments for the network's parameters.

3. Rate of change w.r.t any bias:

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l$$

4. Rate of change w.r.t any weight:

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l$$

7. Improving Accuracy



Weight Decay

Applies L2 Regularization to penalize massive weights. Shrinks parameters to prevent the model from memorizing noise (overfitting).



Dropout

Randomly zeroes out a percentage of neurons during training. Forces the network to build robust, redundant pathways.



Weight Initialization

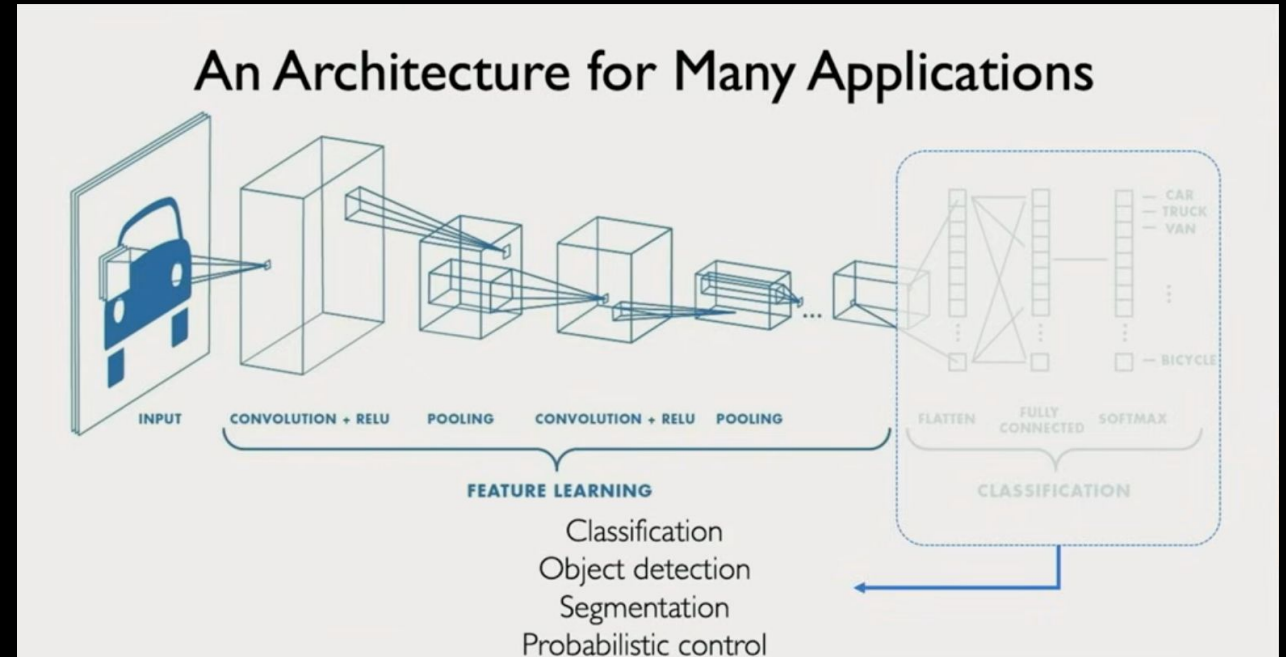
Initializing weights by $w/\sqrt{n}$, preventing vanishing or exploding gradients.

8. Vision: CNNs

Spatial Understanding

Convolutional Neural Networks shift the paradigm from dense, flat linear layers to spatial filters mapping 2D structures.

By sliding kernel matrices across images, CNNs extract hierarchical features: from raw edges to complex shapes..



9. The Adam Optimizer

Momentum Component

Accelerating descent.

- Accumulates velocity in the direction of consistent gradients.
- Dampens oscillations across steep ravines.
- Helps the model power through shallow local minima.

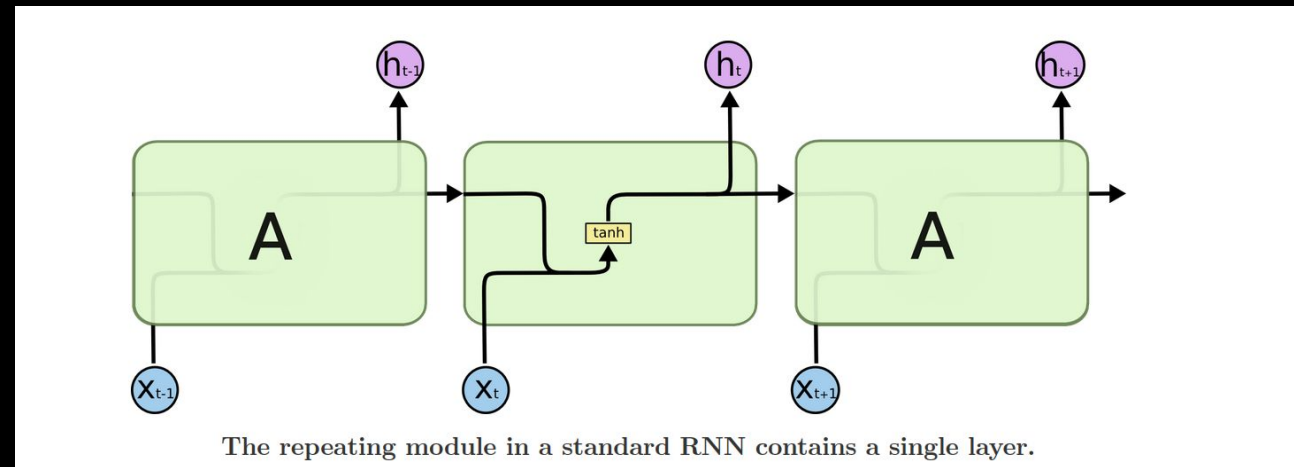
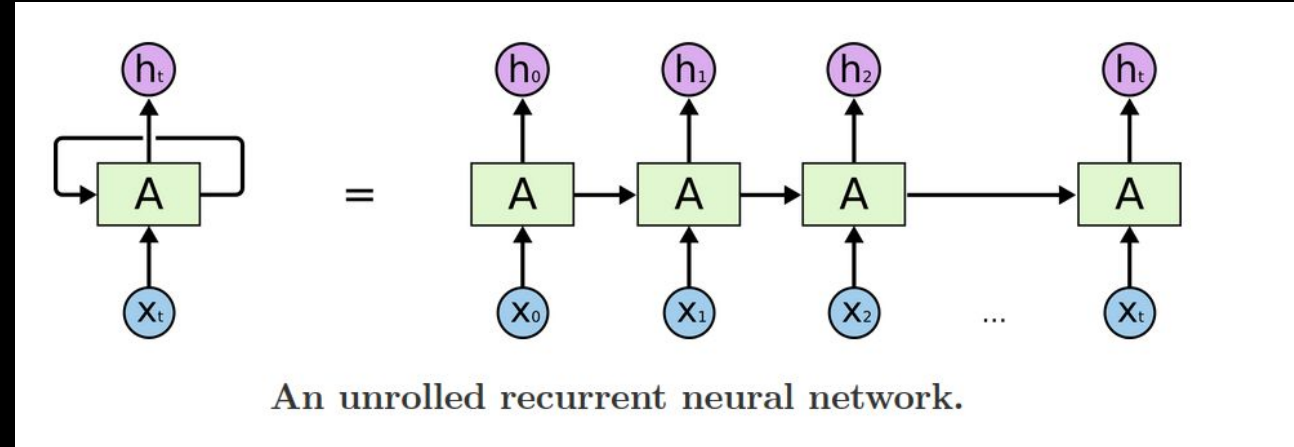
RMSProp Component

Adaptive learning rates.

- Tracks the exponential moving average of squared past gradients.
- Shrinks the learning rate for wildly fluctuating weights.
- Boosts the learning rate for slow, steady weights.

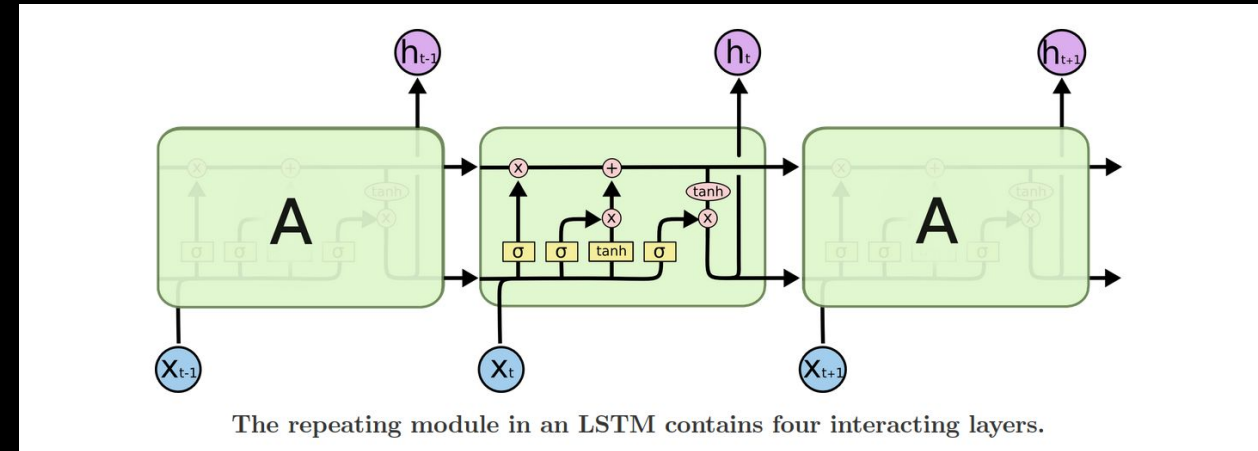
10. Sequences: RNNs

- **Sequential Data:** Built specifically for text, audio, and time-series data.
- **The Snowball Effect:** The hidden state continuously updates, mashing the past and present into a single mathematical summary.
- **Autoregression:** The output of the current timestep becomes the input of the next.
- **The Flaw:** Suffers from catastrophic forgetting over long sequences due to gradient vanishing.



11. Handling long-term dependencies: **LSTMs**

- **Cell State:** The uninterrupted conveyor belt running straight through the entire network, carrying long-term memory safely forward.
- **Forget & Input Gates:** The Forget gate erases irrelevant history, while the Input gate pushes vital new information onto the conveyor belt.
- **Output Gate:** Looks at the massive long-term conveyor belt and reveals only the small piece needed to predict the next step.



12. LSTM Mathematical Formulation

1. The Forget Gate

Decides what information to discard.

$$f_t = \sigma (W_f \cdot [h_{t-1} , x_t] + b_f)$$

2. The Input Gate

Decides which values to update.

$$i_t = \sigma (W_i \cdot [h_{t-1} , x_t] + b_i)$$

3. The Cell Candidate

Creates a vector of new candidate values.

$$\tilde{C}_t = \tanh (W_C \cdot [h_{t-1} , x_t] + b_C)$$

4. Cell State Update

Updates the old cell state into the new cell state.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

5. The Output Gate

Decides what parts of the cell state to output.

$$o_t = \sigma (W_o \cdot [h_{t-1} , x_t] + b_o)$$

6. The Hidden State

Filters the cell state to produce the final output.

$$h_t = o_t * \tanh (C_t)$$

Thank You

